Build your first AI agent

One canteen. Two tools. Your first agent.

Vedant · VIT Mumbai · 17 October 2026

You have Rs 80 and ten minutes

Find vegetarian lunch.
Is the canteen still open?

What information would the program need?

What makes this an agent?

Goal	Help someone find lunch.
Choices	Choose an allowed next action.
Tools	Use a menu or hours lookup.

An AI program can choose steps and use tools toward a goal.

Act out the tool call

Student	Asks for lunch under Rs 80.
Model	Requests a menu lookup.
Tool	Returns matching menu records.

The model requests. Application code executes.

Give each part one job

User asks → Model chooses → Code runs tool

Tool result returns to the model, which continues or finishes.

Our entire canteen menu

Veg sandwich	Rs 60 · vegetarian · available
Poha / Chicken roll	Rs 35 sold out / Rs 75 non-vegetarian
Dosa	Rs 90 · vegetarian · available

Predict the answer for a vegetarian budget of Rs 80.

A tool starts as a function

```
def find_food(max_price: int, vegetarian: bool = True):
    """Find available food within a budget."""
    matches = []
    for dish in MENU:
        # Check stock, budget and diet.
        ...
    return {"status": "ok", "matches": matches}

print(find_food(80))
```

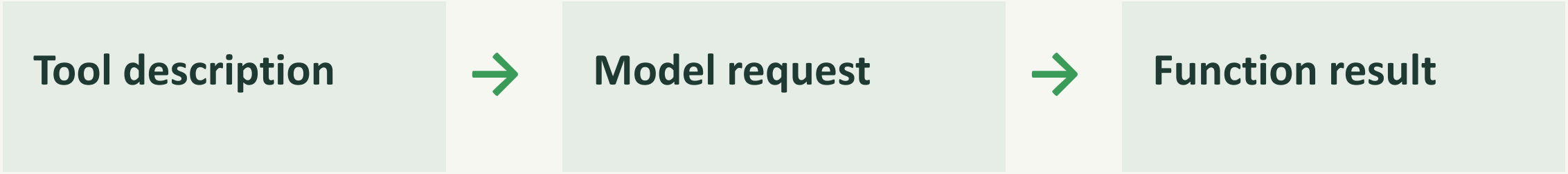
This outline is ordinary Python. Complete it in the editor.

Your turn: run the tool

Create menu_tools.py
Call find_food(80)
Then try find_food(30)

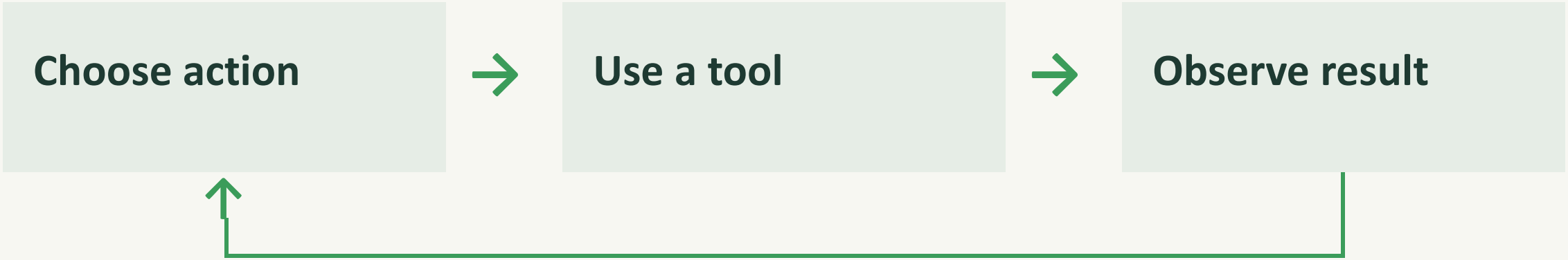
Checkpoint 1: a real local function result

Let the model request the function



The application executes the requested function.

The ReAct-style loop



Continue, use another tool, or finish within a limit.

Build with LangChain

```
agent = create_agent(
    model=make_model(OFFLINE),
    tools=[tool(find_food)],
    system_prompt="Use the menu for facts.",
)
result = agent.invoke(
    {"messages": [("user", "Veg lunch under Rs 80?")]},
    {"recursion_limit": 10},
)
```

Add imports and setup from classroom/README.md.

Build and run your first agent

Write my_agent.py
Register your menu function
Ask for lunch under Rs 80

Checkpoint 2: explain one tool request and its result

What happened in that run?

Request	find_food with a budget and diet
Execution	Python reads and filters our menu
Answer	The model receives the actual result

Try a different question and observe the tool use.

Add one new ability

```
def get_canteen_hours(at: str = "12:30") -> dict:
    """Read fictional hours at a mock time."""
    # See the full short function in the lesson.
    ...

tools=[tool(find_food), tool(get_canteen_hours)]
```

Registering a new tool gives the agent another allowed action.

Your agent now has two tools

Find vegetarian lunch under Rs 80 and tell me when the canteen closes

Checkpoint 3: run the capability you added

Five-minute break

Back at 12:00
Keep your first agent open

Next: reuse its tools through MCP

Can another app use our tools?

We already wrote the menu logic.
How can another agent reuse it?

MCP gives applications a common tool interface.

Same data, same tool service

Clients	LangChain, a LangGraph agent, CrewAI
MCP server	find_food and get_canteen_hours
Data	The same fictional menu and clock

Framework connections differ. The tool contract stays the same.

Expose our functions through MCP

```
from mcp.server.fastmcp import FastMCP
from menu_tools import find_food, get_canteen_hours

server = FastMCP("Campus Canteen")
server.tool()(find_food)
server.tool()(get_canteen_hours)

if __name__ == "__main__":
    server.run(transport="stdio")
```

Use the same functions. Keep server stdout for protocol traffic.

Discover, call, then connect an agent

```
async with connected_tools() as tools:
    print([t.name for t in tools])
    # The client has real MCP tool wrappers.
    agent = create_agent(make_model(OFFLINE), tools)
    result = await agent.ainvoke(
        {"messages": [("user", "Veg lunch under Rs 80?")]}
    )
```

Prepared connection setup lets us focus on the tool boundary.

Try one real MCP call

Discover the two tools
Call find_food with budget 80
Compare with your direct function

Checkpoint 4: same records across a process boundary

What if the canteen is closed?

We want to check hours first.
If closed, skip the lunch lookup.

Now we have a reason to control the route.

One branch around an agent

```
START -> check hours

    open   -> lunch agent -> END
    closed -> closed reply -> END

State = information carried forward
Node  = one step
Edge  = the next route
```

LangGraph can combine explicit steps and flexible agent loops.

Wire the small graph

```
flow = StateGraph(State)
flow.add_node("hours", hours)
flow.add_node("agent", agent)
flow.add_node("closed", closed)
flow.add_edge(START, "hours")
flow.add_conditional_edges(
    "hours", choose, {"agent": "agent", "closed": "closed"}
)
flow.add_edge("agent", END)
flow.add_edge("closed", END)
```

The graph file defines three steps and one branch.

Predict both routes, then run

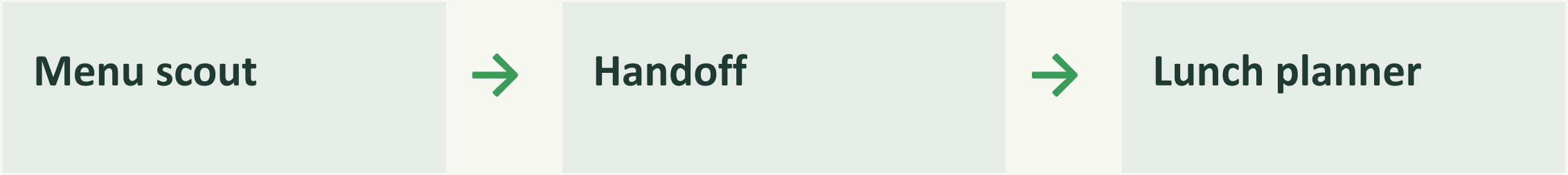
```
python classroom/graph.py --offline

# Simulated later time:
python classroom/graph.py --offline --at 17:00

12:30 -> hours -> agent
17:00 -> hours -> closed
```

The closed route skips the lunch agent.

Could two agents share the job?



One role gathers facts. The next uses those facts.

A real CrewAI handoff

```
lookup = Task(..., agent=scout)
plan = Task(..., agent=planner, context=[lookup])

team = Crew(
    agents=[scout, planner],
    tasks=[lookup, plan],
    process=Process.sequential,
)
team.kickoff()
```

context passes the first task output into the next task.

Run the scout and planner

```
python classroom/crew.py --offline

SCOUT: available options from MCP
PLANNER: 2 x Veg sandwich
TOTAL: Rs 120 within Rs 150
```

Actual CrewAI tasks; --offline uses scripted brains.

When does a team earn its cost?

Useful split	Distinct expertise or responsibilities
Added cost	More model calls and coordination
First question	Would one agent be enough?

An orchestrator assigns work; a sequence has a fixed handoff.

Choose for the job

A few tools	Simple loop or LangChain agent
Required routes	LangGraph workflow with agents where useful
Distinct roles	Consider a CrewAI team and its cost

These are starting points, not exclusive framework capabilities.

MCP and A2A connect different things

MCP	Application <-> tools and context
A2A	Independent agent systems <-> each other
Our crew	Two agents inside one application

Two agents do not automatically implement the A2A protocol.

Give the files clear jobs

```
README.md           what it does and how to run
AGENTS.md           supported coding instructions
data/menu.json      fictional records
menu_tools.py       ordinary functions
my_agent.py         model + tools + loop
```

Instruction filename support depends on your coding client.

Write a useful project instruction

```
# Canteen project rules

- Use data/menu.json for menu facts.
- Explain a proposed change before editing.
- Run the changed behavior and report the result.
- Keep API keys out of source and commits.
```

A heading and clear bullets are enough to start.

Keep control of allowed actions

```
if call["name"] not in allowed_tools:
    raise ValueError("Tool is not allowed")

# Also bound the number of model steps.
# Validate arguments before execution.
# Keep private configuration off screen.
```

Prompts guide behavior. Code enforces boundaries.

Review one small AI-assisted change

Describe	Add a canteen-location tool.
Inspect	Read the new function and registration.
Run	Ask a question that needs the tool.

Ask what changed, why it works and what actually ran.

What will you build next?

One new tool
One graph route
Or compare one agent with a team

Questions · repository and exercises in your student kit